Rising Water Prediction System

Project Description:

Rising Water Prediction System:

The Rising Water Prediction System is a web-based application developed to predict the possibility of rising water levels based on environmental parameters such as Temperature, Humidity, and Cloud Cover. The application helps users quickly determine whether there is a chance of water levels rising by analyzing the given input values through a prediction model.

The system is developed using the Flask framework for the backend and HTML, CSS, and JavaScript for the frontend, providing a simple, responsive, and user-friendly interface. Users enter the required weather parameters, and the application processes the inputs to display one of two prediction results: Chance of Rising Water or No Chance of Rising Water.

The application is deployed locally using Flask and can be shared publicly using Ngrok. It demonstrates how weather-related environmental data can be used in a simple prediction system, making it useful for educational purposes and as a foundation for future flood monitoring applications.

Scenarios

Scenario 1:

A user enters Temperature = 20°C, Humidity = 62%, and Cloud Cover = 20%. The system analyzes the values and predicts Chance of Rising Water, displaying the corresponding result page.

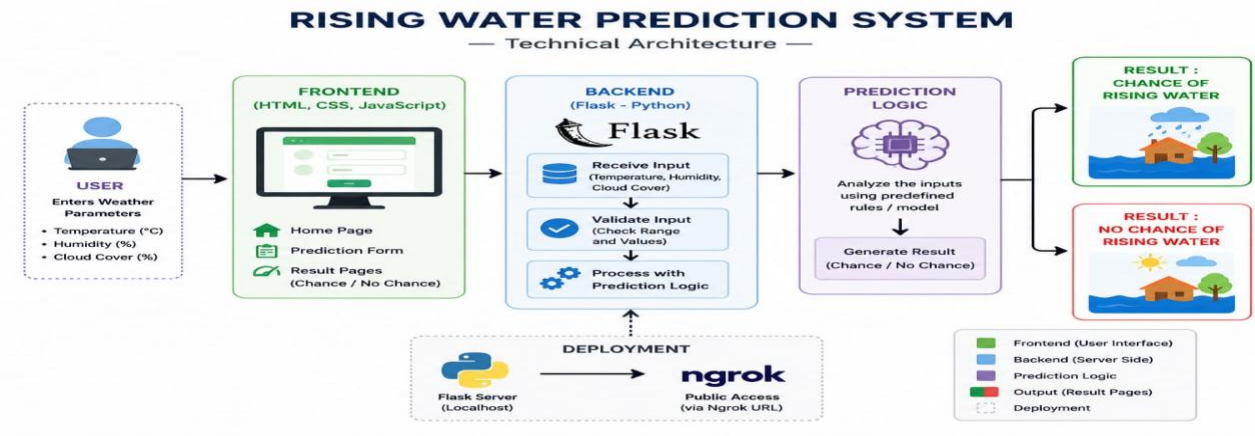
Scenario 2:

A user enters Temperature = 35°C, Humidity = 40%, and Cloud Cover = 10%. Based on the prediction logic, the application displays No Chance of Rising Water.

Scenario 3:

A user tests multiple combinations of weather parameters to understand how different environmental conditions affect the prediction. The system instantly displays the appropriate prediction result, helping users evaluate different weather situations.

Technical Architecture:



Description: The Rising Water Prediction System follows a modular three-tier architecture to provide a simple and efficient water level prediction service. The frontend is developed using HTML, CSS, and JavaScript to provide an interactive and responsive user interface. The

backend is built using the Flask framework, which receives user inputs, processes the prediction logic, and returns the prediction result.

The application accepts three environmental parameters: Temperature, Humidity, and Cloud Cover. These values are validated by the Flask backend and processed using the project's prediction logic. Based on the analysis, the system determines whether there is a Chance of Rising Water or No Chance of Rising Water and displays the corresponding result page.

The project is deployed locally using the Flask development server and can be shared publicly using Ngrok. The architecture is lightweight, easy to maintain, and suitable for educational and demonstration purposes.

(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites:

1.Flask Framework Knowledge
2.Python Programming Basics
3.HTML, CSS, and JavaScript Knowledge
4.Visual Studio Code (or any Python IDE)
5.Python 3.8 or above
6.pip Package Manager
7.Git and GitHub for Version Control
8.Flask Development Server
9.Ngrok for Public Deployment
10.Basic understanding of Weather Parameters such as Temperature, Humidity, and Cloud Cover

Project Workflow

Milestone 1: Project Planning and Architecture

Activity 1.1: Define Project Requirements

- Identify the objective of predicting rising water levels using weather parameters.

- Select the input parameters: Temperature, Humidity, and Cloud Cover.

- Define the expected outputs: Chance of Rising Water or No Chance of Rising Water.

Activity 1.2: Design the System Architecture

- Design the interaction between the user, frontend, backend, and prediction logic.

- Plan the flow of data from user input to prediction result.

Activity 1.3: Set Up the Development Environment

- Install Python 3.8 or above.

- Install Flask using pip.

- Configure Visual Studio Code.

- Create the project folders (templates, static, app.py).

Milestone 2: Core Functionality Development

Activity 2.1: Develop Prediction Logic

- Accept Temperature, Humidity, and Cloud Cover as inputs.

- Validate all input values.

- Apply the prediction logic to determine the possibility of rising water.

Activity 2.2: Implement Flask Backend

- Create Flask routes.

- Receive user inputs.

- Process the prediction.

- Return the appropriate result page.

Milestone 3: Frontend Development

Activity 3.1: Design User Interface

- Develop Home Page.

- Create Prediction Form.

- Add responsive design using HTML and CSS.

Activity 3.2: Display Prediction Results

- Show Chance of Rising Water when prediction conditions are satisfied.

- Show No Chance of Rising Water when prediction conditions are not satisfied

Milestone 4: Testing

Activity 4.1: Test Different Input Values

- Verify predictions using multiple combinations of Temperature, Humidity, and Cloud Cover.

- Ensure the application displays the correct result page.

Activity 4.2: Fix Errors

- Identify validation errors.

- Improve application performance and user experience.

Milestone 5: Deployment

Activity 5.1: Local Deployment

- Run the Flask application on localhost.

- Verify that all pages work correctly.

Activity 5.2: Public Deployment

- Upload the project to GitHub.

- Deploy using Ngrok.

- Share the public URL for testing and demonstration.

Milestone 6: Conclusion

- Successfully develop and deploy the Rising Water Prediction System.

- Demonstrate accurate prediction based on user-provided weather parameters.

- Provide a user-friendly application for educational and demonstration purposes.

Milestone 1: Prediction Model Selection and System Architecture

In this milestone, the prediction approach and system architecture for the Rising Water Prediction System are defined. The application predicts the possibility of rising water using environmental parameters such as Temperature, Humidity, and Cloud Cover. The development environment and project structure are also prepared.

Activity 1.1: Select the Prediction Model

The Rising Water Prediction System uses a rule-based prediction model instead of a machine learning model. The prediction is based on predefined conditions using Temperature, Humidity, and Cloud Cover values.

The model performs the following tasks:

- Accept Temperature, Humidity, and Cloud Cover as inputs.

- Validate the input values.

- Apply predefined prediction rules.

- Display one of the following results:

  - Chance of Rising Water

  - No Chance of Rising Water

The rule-based model was selected because it is simple, fast, easy to implement, and suitable for educational demonstrations.

Activity 1.2: Define the System Architecture

- Design the interaction between the user, frontend, backend, and prediction logic.

- Develop the frontend using HTML, CSS, and JavaScript.

- Implement the backend using Flask (Python).

- Process user inputs through the prediction logic and display the prediction result.

Activity 1.3: Set Up the Development Environment

- Install Python and Flask.

- Create the project structure.

- Configure Visual Studio Code.

- Create the templates and static folders.

- Develop the Flask application (app.py).

- Test the application locally before deployment.

Milestone 2: Core Functionalities Development

Milestone 2 focuses on developing the core functionalities of the Rising Water Prediction System. In this phase, the Flask backend is implemented to receive user inputs, validate the weather parameters, execute the prediction logic, and display the prediction result.

Activity 2.1: Develop the Prediction Functionality

The core functionality of the Rising Water Prediction System is to predict the possibility of rising water levels using Temperature, Humidity, and Cloud Cover values.

The system performs the following operations:

- Accept Temperature, Humidity, and Cloud Cover as user inputs.

- Validate that all input fields contain valid numeric values.

- Process the values using the prediction logic implemented in the Flask application.

- Determine whether there is a Chance of Rising Water or No Chance of Rising Water.

- Display the corresponding result page to the user.

Features

- User-friendly prediction form.

- Fast prediction processing.

- Input validation.

- Separate result pages for positive and negative predictions.

- Responsive web interface.

Activity 2.2: Implement the Flask Backend

The Flask backend manages all application routes and prediction processing.

The backend performs the following tasks:

- Define the application routes.

- Receive user inputs from the prediction form.

- Validate Temperature, Humidity, and Cloud Cover values.

- Execute the prediction logic.

- Redirect users to the appropriate result page.

Flask Routes

- Home Route (/) – Displays the Home Page.

- Prediction Route (/predict) – Receives user inputs and processes the prediction.

- Chance Route – Displays the "Chance of Rising Water" page.

- No Chance Route – Displays the "No Chance of Rising Water" page.

The backend ensures that every prediction request is processed correctly and that the appropriate result page is displayed based on the entered weather parameters.

Milestone 3: App.py Development

Milestone 3 focuses on developing the main application logic in the app.py file. The Flask application handles user requests, validates the input values, executes the prediction logic, and displays the appropriate prediction result.

Activity 3.1: Writing the Main Application Logic in app.py

The app.py file is the core component of the Rising Water Prediction System. It connects the frontend with the backend and controls the complete prediction process.

Define the Flask Routes

The application contains the following routes:

- / – Displays the Home page.

- /predict – Receives Temperature, Humidity, and Cloud Cover values from the user.

- Processes the prediction logic.

- Displays either Chance of Rising Water or No Chance of Rising Water.

Input Validation

The Flask application validates the user inputs before processing the prediction.

Validation includes:

- Temperature should be a valid numeric value.

- Humidity should be a valid numeric value.

- Cloud Cover should be a valid numeric value.

- Empty input fields are not accepted.

If invalid data is entered, the application prompts the user to enter valid values.

Prediction Processing

After successful validation, the application processes the entered values using the predefined prediction logic.

Based on the processed data, the application determines whether:

- Chance of Rising Water

- No Chance of Rising Water

The corresponding result page is then displayed.

Response Generation

The backend sends the prediction result to the frontend, where the user can immediately view the output.

The application ensures:

- Fast response time.

- Accurate processing of user inputs.

- Simple and user-friendly interaction.

- Reliable prediction result display.

This implementation makes the Rising Water Prediction System efficient, lightweight, and easy to maintain.

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing a responsive and user-friendly interface for the Rising Water Prediction System. The frontend is built using HTML, CSS, and JavaScript to provide an interactive experience for users.

Activity 4.1: Designing and Developing the User Interface

The frontend of the application consists of multiple web pages that allow users to enter weather parameters and view prediction results.

Home Page

- Displays the project title and introduction.

- Provides a navigation button to access the prediction page.

-The application directly opens the Prediction Page where users can enter weather parameters.

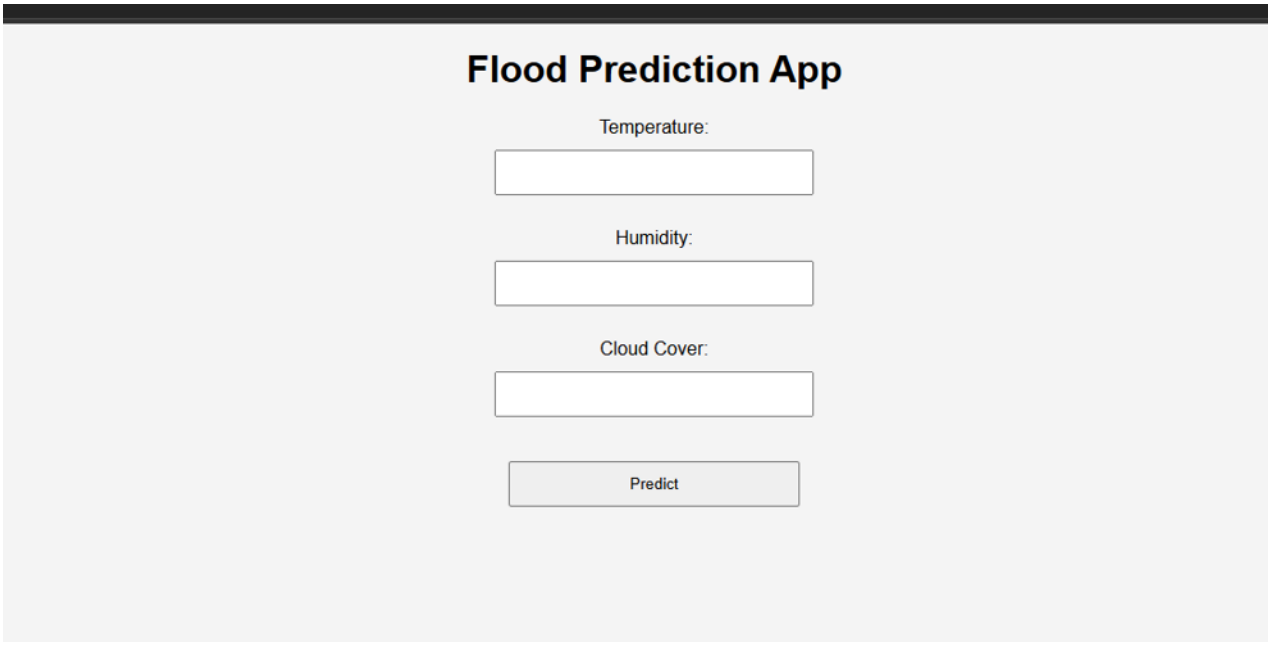
Prediction Page

- Contains a form for entering:

  - Temperature (°C)

  - Humidity (%)

  - Cloud Cover (%)



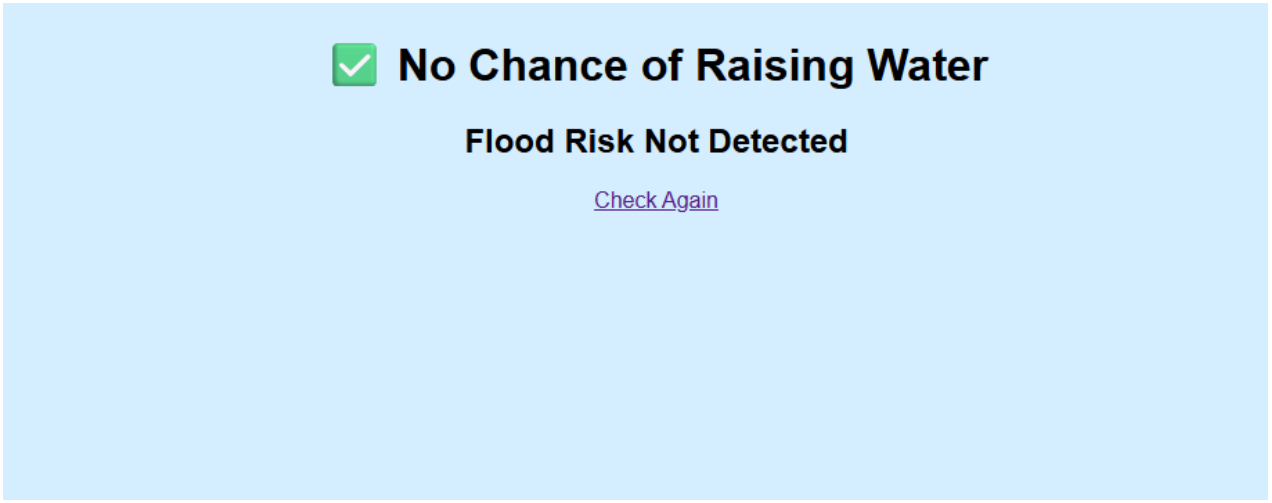
- Includes a Predict button to submit the data.

Result Pages

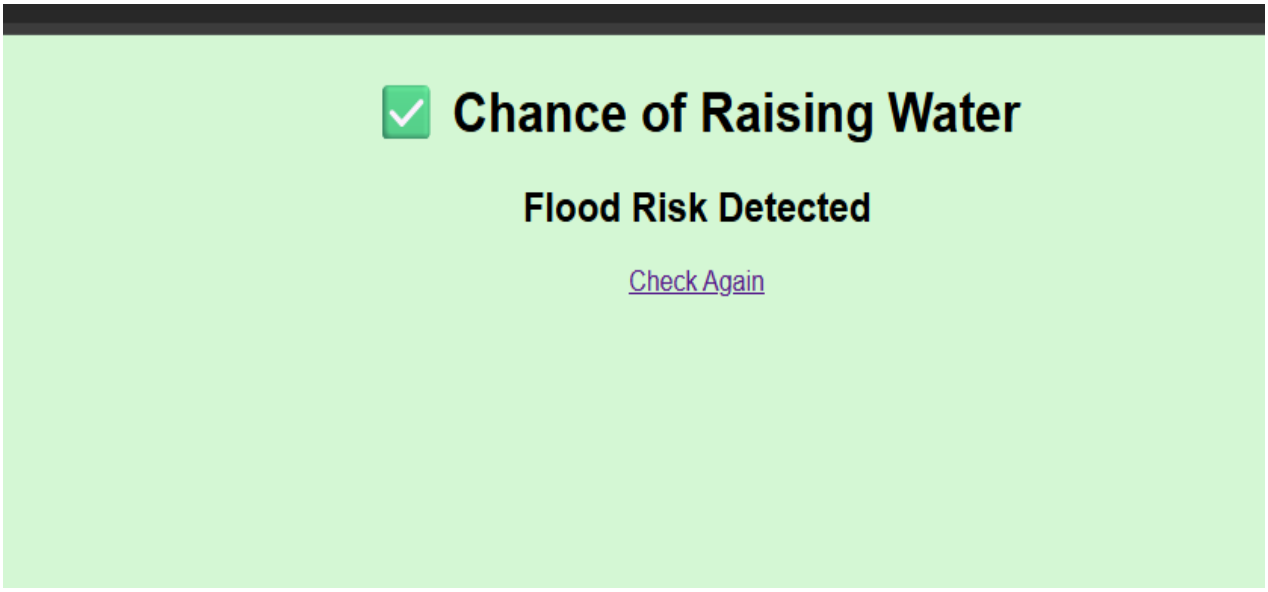
- Chance of Rising Water: Displays when the prediction indicates a possibility of rising water.

- No Chance of Rising Water: Displays when the prediction indicates no possibility of rising water.

The interface is designed to be simple, responsive, and easy to use on desktop and mobile devices.



-The interface is designed to be simple, responsive, and easy to use on desktop and mobile devices.



Activity 4.2: Dynamic Content Rendering

The application uses JavaScript to improve user interaction.

The frontend performs the following tasks:

- Accepts user input through HTML forms.

- Sends the input data to the Flask backend.

- Receives the prediction result.

- Displays the appropriate result page.

- Provides a smooth and responsive user experience.

Frontend Features

- Responsive web design.

- Simple navigation between pages.

- User-friendly input forms.

- Attractive styling using CSS.

- Fast display of prediction results.

The frontend works seamlessly with the Flask backend to provide a complete Rising Water Prediction System.

Milestone 5: Deployment

Milestone 5 focuses on deploying the Rising Water Prediction System and making it available for testing. The application is first tested on the local machine using the Flask development server. After successful testing, the project is deployed online to enable public access.

Activity 5.1: Local Deployment

The Rising Water Prediction System is tested locally to ensure that all functionalities work correctly before deployment.

Steps

Open the project in Visual Studio Code.

Open the integrated terminal.

Run the Flask application using the following command:

py app.py

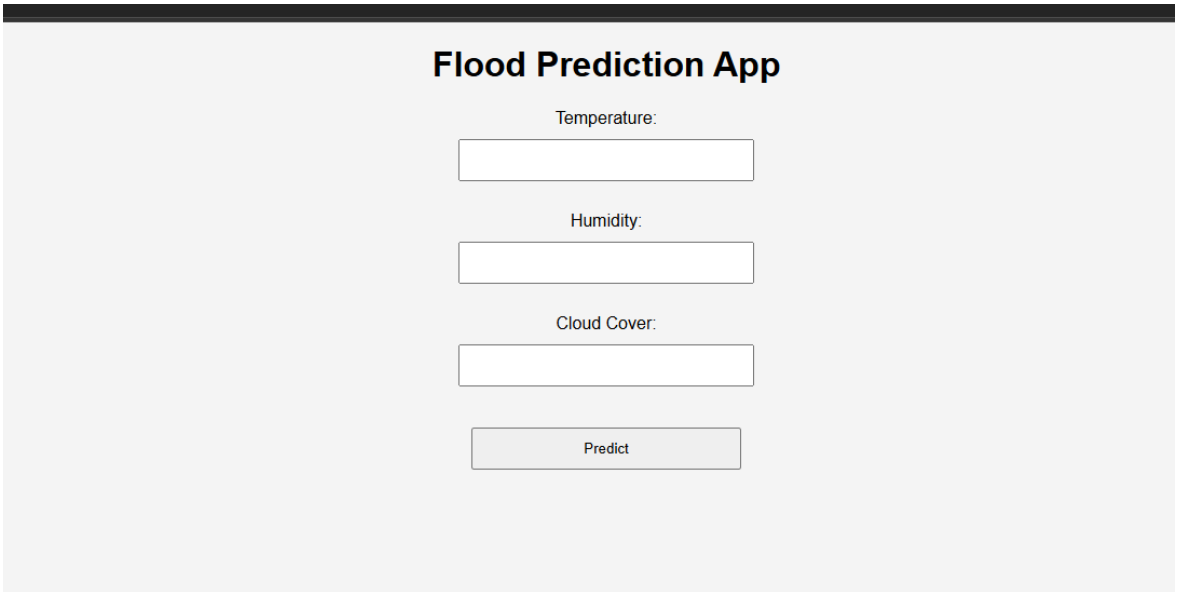
Open the browser and navigate to:

http://127.0.0.1:5000

Verify that the Prediction Page loads successfully.

Test the application with different weather parameters

```
PS C:\Users\satya\OneDrive\Desktop\Rising water> py app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI ser
ver instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 461-491-814
PS C:\Users\satya\OneDrive\Desktop\Rising water>
```

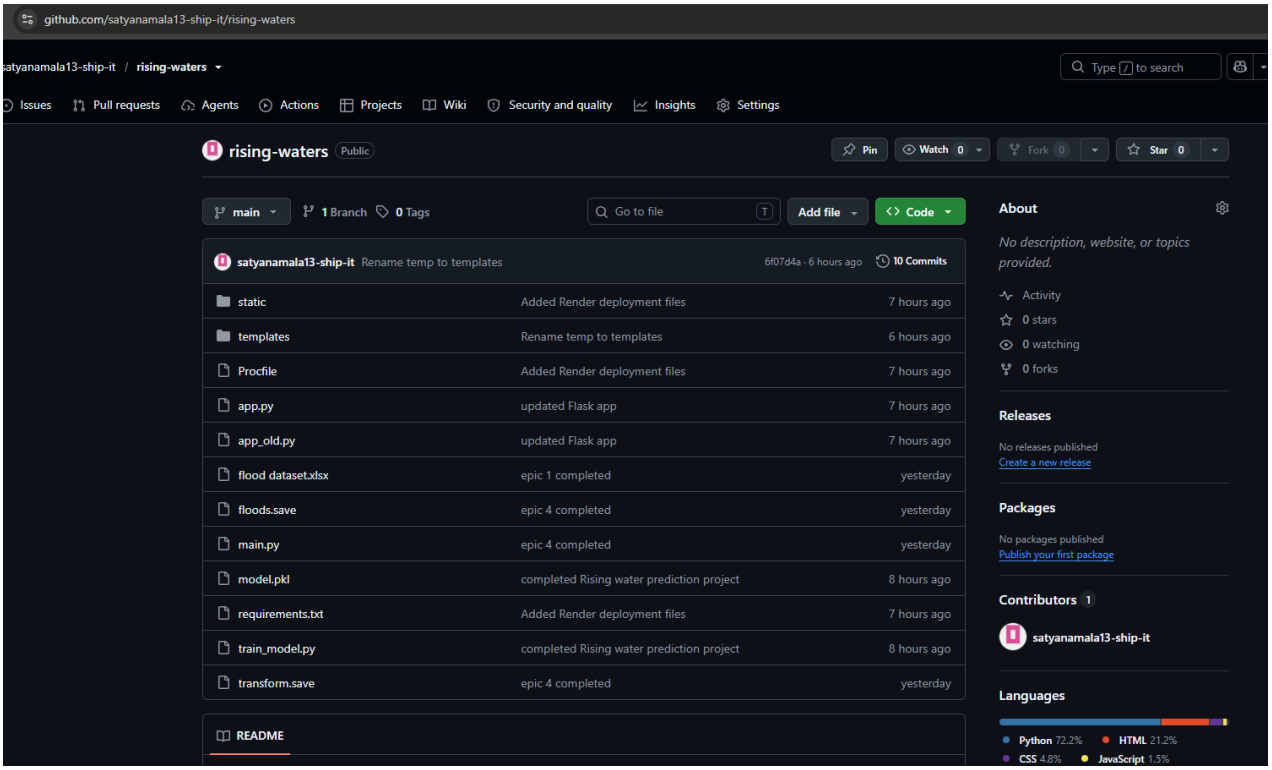


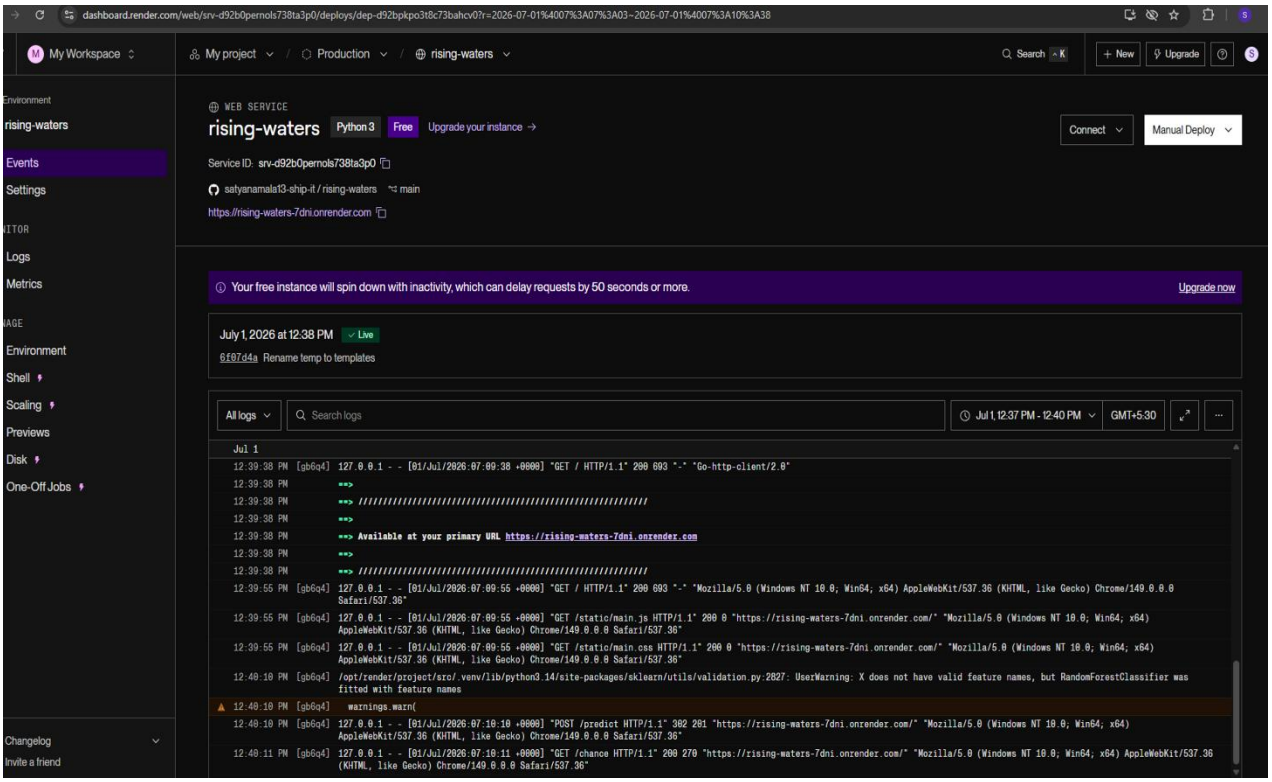
Activity 5.2: Public Deployment

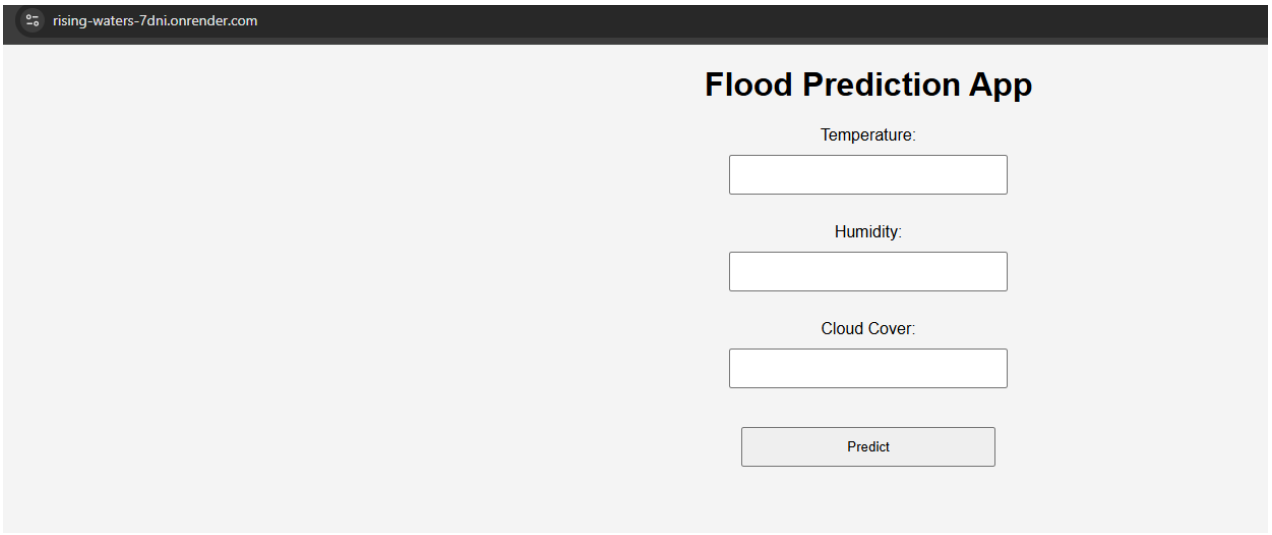
After successful local testing, the project is deployed online.

Steps

- Push the project to GitHub.
- Deploy the application using **Render** (or **Ngrok**, if that's what you used).
- Verify that the live application opens successfully.
- Share the public URL for testing.


github.com/satyanamala13-ship-it/rising-waters
satyanamala13-ship-it / rising-waters
Type / to search
Issues
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
rising-waters
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
Add file
Code
satyanamala13-ship-it Rename temp to templates
6f07d4a · 6 hours ago
10 Commits
static Added Render deployment files 7 hours ago
templates Rename temp to templates 6 hours ago
Procfile Added Render deployment files 7 hours ago
app.py updated Flask app 7 hours ago
app_old.py updated Flask app 7 hours ago
flood dataset.xlsx epic 1 completed yesterday
floods.save epic 4 completed yesterday
main.py epic 4 completed yesterday
model.pkl completed Rising water prediction project 8 hours ago
requirements.txt Added Render deployment files 7 hours ago
train_model.py completed Rising water prediction project 8 hours ago
transform.save epic 4 completed yesterday
README
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
satyanamala13-ship-it
Languages
Python 72.2%
HTML 21.2%
CSS 4.8%
JavaScript 1.5%


dashboard.render.com/web/srv-d92b0pernols738ta3p0/deploys/dep-d92bpkpo3t8c73bahcv0?r=2026-07-01%4007%3A07%3A03~2026-07-01%4007%3A10%3A38
My Workspace
My project
Production
rising-waters
Search
New
Upgrade
Environment
rising-waters
Events
Settings
Logs
Metrics
Environment
Shell
Scaling
Previews
Disk
One-Off Jobs
Changelog
Invite a friend
WEB SERVICE
rising-waters
Python 3
Free
Upgrade your instance →
Connect
Manual Deploy
Service ID: srv-d92b0pernols738ta3p0
satyanamala13-ship-it / rising-waters
main
https://rising-waters-7dni.onrender.com
Your free instance will spin down with inactivity, which can delay requests by 50 seconds or more.
Upgrade now
July 1, 2026 at 12:38 PM
Live
6f07d4a Rename temp to templates
All logs
Search logs
Jul 1, 12:37 PM - 12:40 PM
GMT+5:30
Jul 1
12:39:38 PM [gb6q4] 127.0.0.1 - - [01/Jul/2026:07:09:38 +0000] "GET / HTTP/1.1" 200 693 "-" "Go-http-client/2.0"
12:39:38 PM ==>
12:39:38 PM ==> ///
12:39:38 PM ==>
12:39:38 PM ==> Available at your primary URL https://rising-waters-7dni.onrender.com
12:39:38 PM ==>
12:39:38 PM ==> ///
12:39:55 PM [gb6q4] 127.0.0.1 - - [01/Jul/2026:07:09:55 +0000] "GET / HTTP/1.1" 200 693 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
12:39:55 PM [gb6q4] 127.0.0.1 - - [01/Jul/2026:07:09:55 +0000] "GET /static/main.js HTTP/1.1" 200 0 "https://rising-waters-7dni.onrender.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
12:39:55 PM [gb6q4] 127.0.0.1 - - [01/Jul/2026:07:09:55 +0000] "GET /static/main.css HTTP/1.1" 200 0 "https://rising-waters-7dni.onrender.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
12:40:10 PM [gb6q4] /opt/render/project/src/.venv/lib/python3.14/site-packages/sklearn/utils/validation.py:2027: UserWarning: X does not have valid feature names, but RandomForestClassifier was fitted with feature names
12:40:10 PM [gb6q4] warnings.warn(
12:40:10 PM [gb6q4] 127.0.0.1 - - [01/Jul/2026:07:10:10 +0000] "POST /predict HTTP/1.1" 302 201 "https://rising-waters-7dni.onrender.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"
12:40:11 PM [gb6q4] 127.0.0.1 - - [01/Jul/2026:07:10:11 +0000] "GET /chance HTTP/1.1" 200 270 "https://rising-waters-7dni.onrender.com/" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36"



Conclusion

The **Rising Water Prediction System** was successfully designed, developed, and deployed using Machine Learning and Flask. The system predicts the possibility of rising water levels based on environmental parameters such as rainfall, temperature, humidity, and water level inputs. The trained model provides quick and reliable predictions through a simple web interface, making the application easy to use.

The project demonstrates how Machine Learning can be applied to support early awareness of potential water-related risks. It also highlights the integration of data preprocessing, model training, web development, and deployment into a complete end-to-end application.

Future Scope

The functionality of the Rising Water Prediction System can be enhanced in the future by:

- Integrating real-time weather and water level data from IoT sensors and APIs.
- Improving prediction accuracy using larger and more diverse datasets.
- Implementing advanced Machine Learning and Deep Learning algorithms.
- Sending SMS, email, or mobile notifications when a high-risk prediction is detected.
- Developing an Android or iOS mobile application for easier access.
- Displayin`g prediction results on an interactive map with location-based alerts.

- Supporting multiple languages to make the system accessible to a wider range of users.

- Extending the system to assist disaster management authorities in flood monitoring and emergency planning.